SDG BLOCKCHAIN ACCELERATOR

Prototype (PoC) Report

SolarVillage: A Hydra-based Energy Settlement & Carbon Issuance Architecture

1. Project Information

- **Project Name:** SolarVillage
- **Challenge & UNDP Office:** UNDP Nigeria
- **Document Version:** v1.0 (Jan 27, 2025)

2. Project Overview

SolarVillage is a decentralized energy settlement and carbon accounting platform designed for rural and peri-urban microgrids, starting with pilots in Nigeria.

The project addresses three interrelated problems:

1. **Energy access and payment reliability** in communities with limited banking penetration.
2. **Operational transparency and auditability** for microgrid operators, investors, and regulators.
3. **Credible carbon credit generation** from clean energy delivery, with reduced fraud and double-counting risk.

The system uses **Cardano Hydra** for high-frequency, low-cost operational accounting (metering, tariffs, settlements) and **Cardano Layer 1 (L1)** for low-frequency, high-assurance issuance of carbon assets.

Key features include:

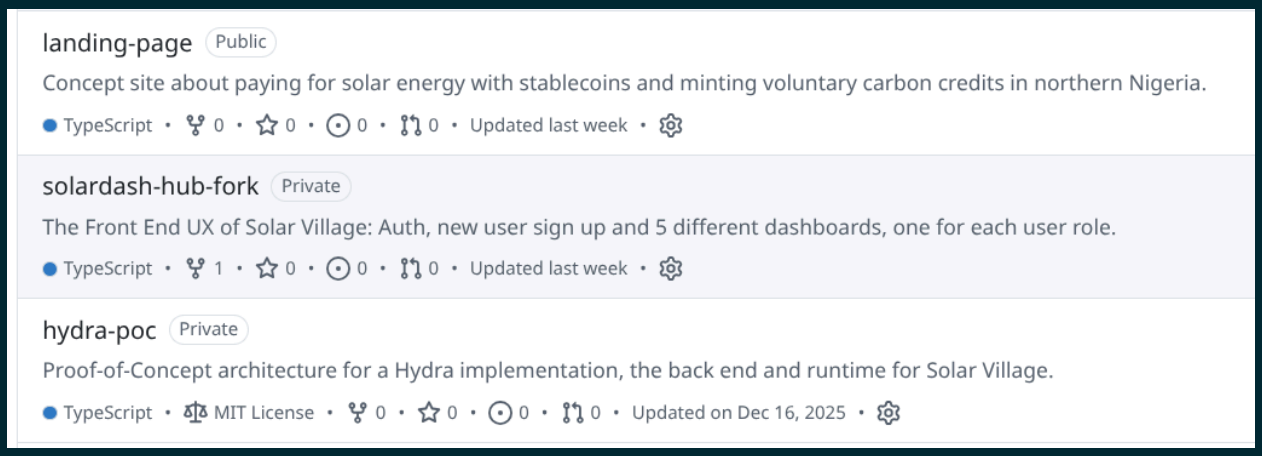
- Custodial consumer balances (no consumer wallets required)
- Real-time settlement in an internal NGN-denominated accounting credit (SVN)
- Deterministic accumulation of delivered energy (WH) for audit and carbon eligibility
- Periodic issuance of **fungible carbon credits (grams CO_2e)** and a **certificate NFT** on Cardano L1

- Alignment with **Verra / Gold Standard–style methodologies** and the forthcoming **Nigerian Carbon Market (NCCC) registry**

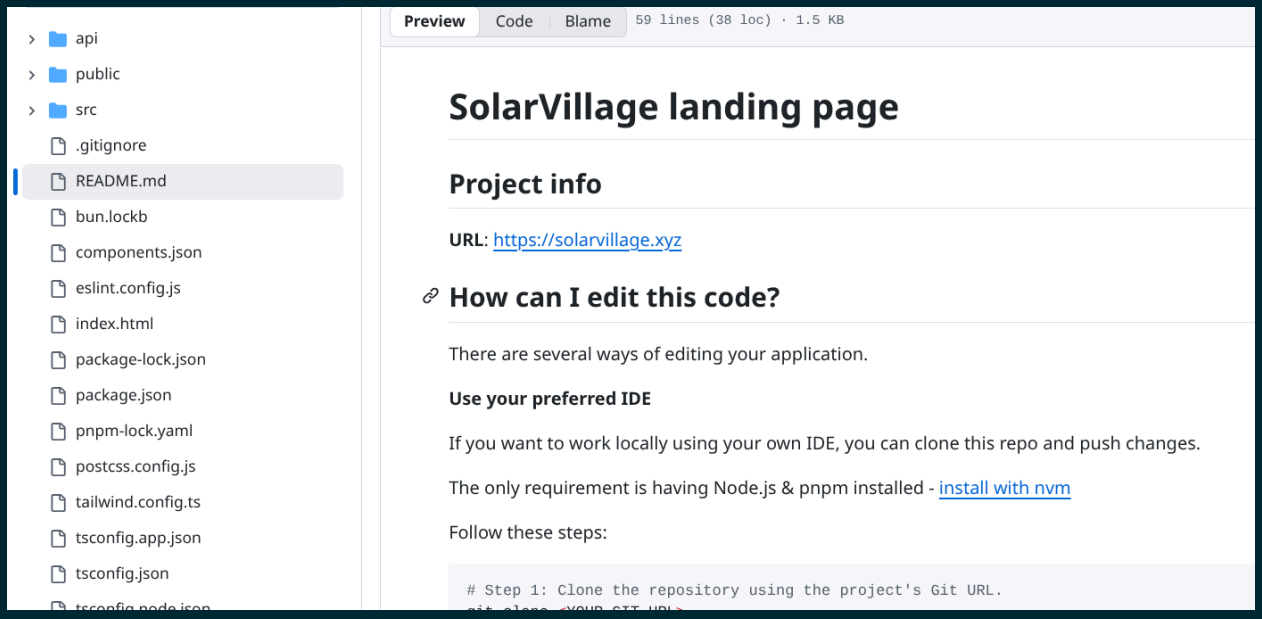
Smart contract logic is implemented primarily in **Aiken**, both for Hydra-side validators and L1 minting policies, with off-chain services coordinating oracle inputs, signatures, and anchoring.

3. Repository Structure

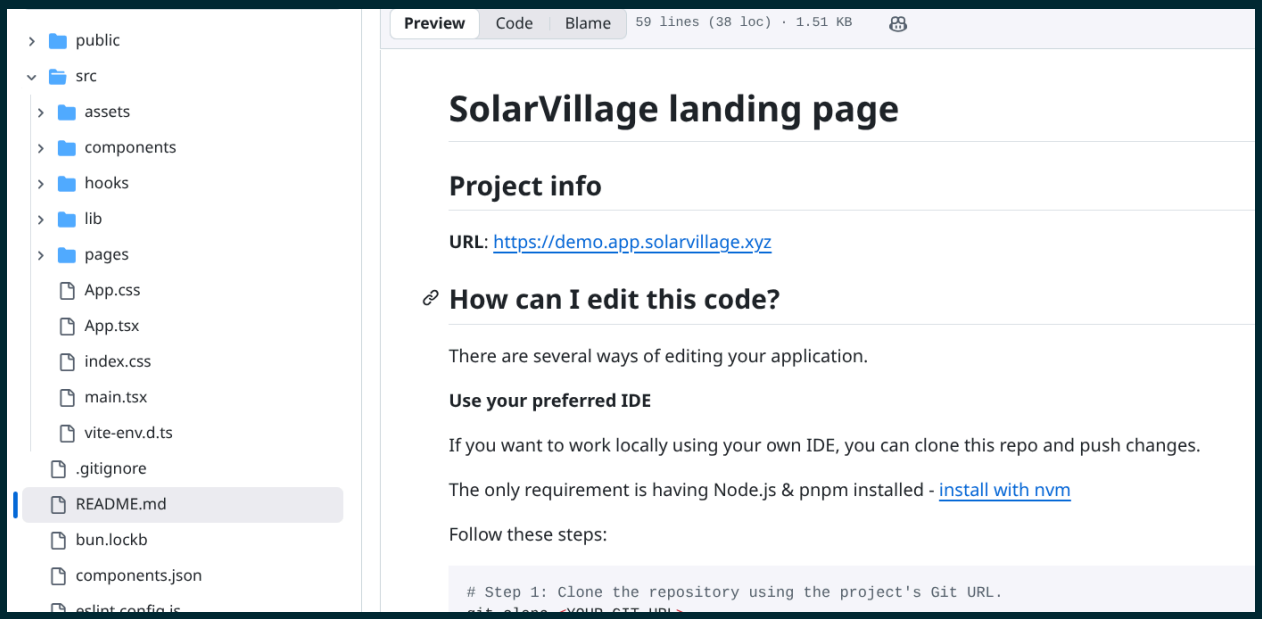
We are maintaining three core repositories:



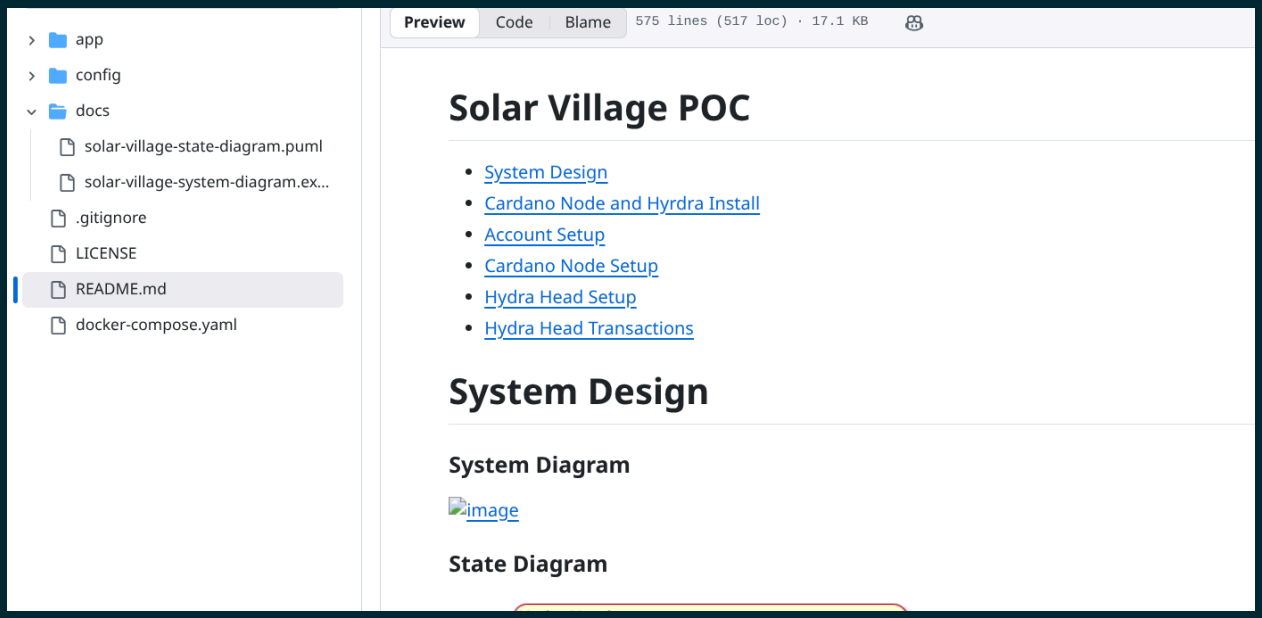
3. 1 Landing page repo structure:

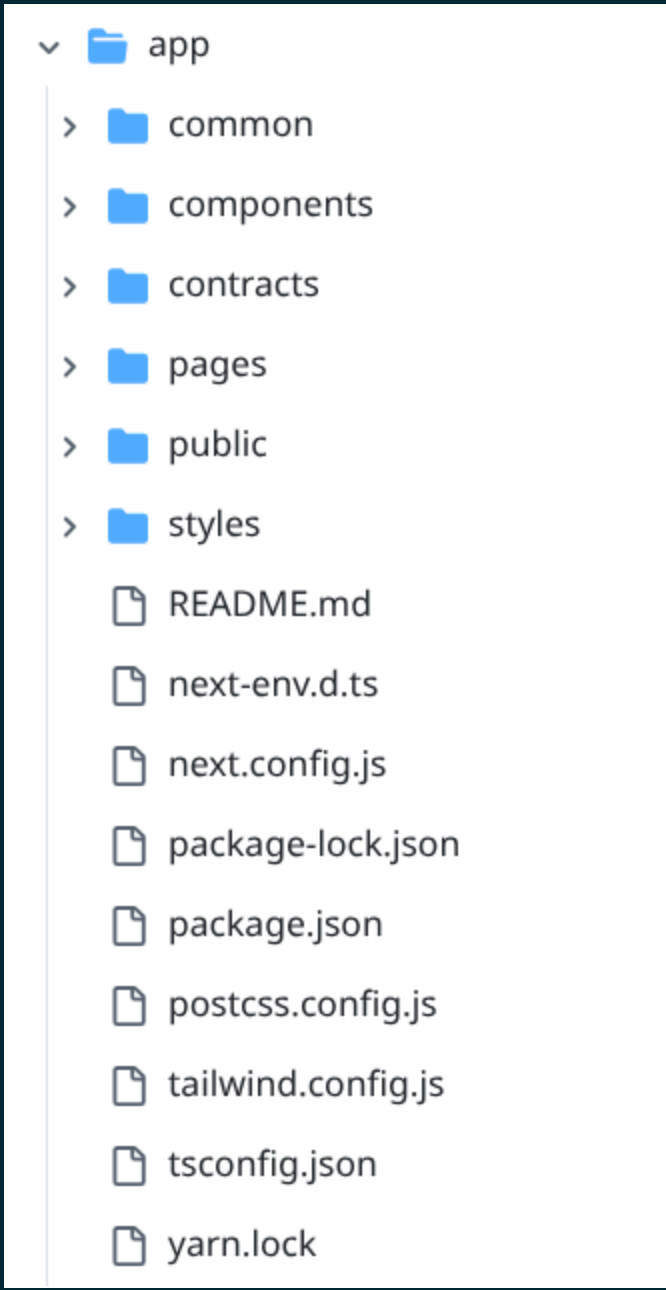


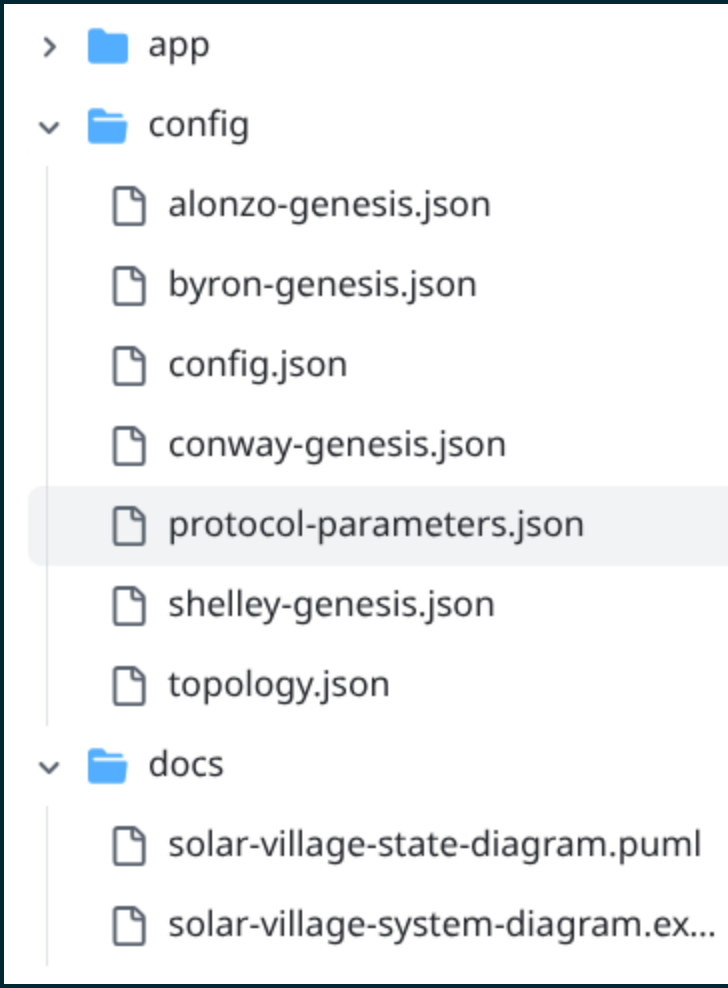
3.2 Dashboard repo structure:



3.3 Hydra runtime repo structure:




app
common
components
contracts
pages
public
styles
README.md
next-env.d.ts
next.config.js
package-lock.json
package.json
postcss.config.js
tailwind.config.js
tsconfig.json
yarn.lock



4. Build Instructions - Front End

Installation & Setup

This project is a standard Vercel-ready web application built with **pnpm**. Follow the steps below to get it running locally for development, testing, or deployment.

Prerequisites

Before you begin, make sure you have the following installed on your machine:

* **Node.js** (recommended: latest LTS version)
* **Git**
* **Corepack** (ships with modern Node.js versions)
* **pnpm** (managed via Corepack)

You can verify your setup with:

```
node -v
git --version
corepack --version
```

Clone the Repository

Start by cloning the repository from GitHub and navigating into the project directory:

```
git clone https://github.com/Solar-Village/solardash-hub-fork
cd solardash-hub-fork
```

Enable pnpm via Corepack

This project uses **pnpm** for dependency management. Enable it using Corepack:

```
corepack enable
```

If pnpm isn't already activated, Corepack will automatically install the correct version specified by the project.

Install Dependencies

Install all required dependencies defined in `package.json`:

```
pnpm install
```

This will:

* Resolve and install dependencies
* Generate a `pnpm-lock.yaml` file (if not already present)
* Prepare the project for local development and builds

Build the Project

To ensure everything compiles correctly, run the production build:

```
pnpm build
```

This step is especially important before deploying to Vercel, as it mirrors the build process used in production.

Lint and Run Tests

To check code quality and verify that tests are passing, run:

```
pnpm lint
pnpm test-unit
```

* **`pnpm lint`** checks formatting and code style issues
* **`pnpm test-unit`** runs the unit test suite

Fix any reported issues before continuing.

Run Locally

If the project includes a development server, you can typically start it with:

```
pnpm dev
```

Then open your browser at the local URL shown in the terminal (commonly `http://localhost:3000`).

Deployment

This project is designed to deploy cleanly on **Vercel**. Once the build passes locally, you can:

* Import the repository into Vercel
* Configure environment variables (if required)
* Deploy directly from the Vercel dashboard or via the Vercel CLI

---

4. Build Instructions - Hydra Heads and Etcd Key-Value Store

To build the Hydra runtime on Cardano follow these instructions (see the [readme.md]() document for up to date instructions):

```
### Download Hydra and etcd Binaries

```
git clone https://github.com/lley154/solar-village.git
mkdir -p bin

hydra_version=1.1.0
curl -L -O
```
```

```
https://github.com/cardano-scaling/hydra/releases/download/${hydra_version}
/hydra-x86_64-linux-${hydra_version}.zip
unzip -d bin hydra-x86_64-linux-${hydra_version}.zip

etcd_version=v3.5.21
curl -L
https://github.com/etcd-io/etcd/releases/download/${etcd_version}/etcd-${et
cd_version}-linux-amd64.tar.gz \
  | tar xz -C bin --strip-components=1

chmod +x bin/*
```

Set ENV variables

```
export PATH=$(pwd)/bin:$PATH
export CARDANO_NODE_SOCKET_PATH=$(pwd)/node.socket
export CARDANO_NODE_NETWORK_ID=1
```

Download latest Mithril Client

```
docker compose -f docker-compose.yaml run mithril-client --origin-tag HYDRA
cardano-db download latest --include-ancillary
```

5. Cardano Node Setup

Startup docker cardano node image

```
docker compose -f docker-compose.yaml up cardano-node
```

Open up a new terminal window and log into the cardano node docker image

```
docker exec -it solar-village-cardano-node-1 bash
```
```
cd /solar-village
```
```
# Confirm that network is 100% sync
cardano-cli query tip
{
    "block": 4116255,
    "epoch": 252,
    "era": "Conway",
    "hash":
"cf4754b443aff2cd70d582d33aa3f24e1243ac22f9dc2cad775269849773fde6",
    "slot": 107317017,
    "slotInEpoch": 94617,
    "slotsToEpochEnd": 337383,
    "syncProgress": "100.00"
}
```

6. Address & Wallet Setup

```
### Create credentials and addresses
```
mkdir -p credentials

cardano-cli address key-gen \
  --verification-key-file credentials/hydra-node.vk \
  --signing-key-file credentials/hydra-node.sk

cardano-cli address build \
  --verification-key-file credentials/hydra-node.vk \
  --out-file credentials/hydra-node.addr
```
```

```
cardano-cli address key-gen \
  --verification-key-file credentials/reserve-wallet.vk \
  --signing-key-file credentials/reserve-wallet.sk

cardano-cli address build \
  --verification-key-file credentials/reserve-wallet.vk \
  --out-file credentials/reserve-wallet.addr

cardano-cli address key-gen \
  --verification-key-file credentials/user-wallet.vk \
  --signing-key-file credentials/user-wallet.sk

cardano-cli address build \
  --verification-key-file credentials/user-wallet.vk \
  --out-file credentials/user-wallet.addr

cardano-cli address key-gen \
  --verification-key-file credentials/meter-wallet.vk \
  --signing-key-file credentials/meter-wallet.sk

cardano-cli address build \
  --verification-key-file credentials/meter-wallet.vk \
  --out-file credentials/meter-wallet.addr

cardano-cli address key-gen \
  --verification-key-file credentials/operator-wallet.vk \
  --signing-key-file credentials/operator-wallet.sk

cardano-cli address build \
  --verification-key-file credentials/operator-wallet.vk \
  --out-file credentials/operator-wallet.addr
```

Send funds to hydra node and reserve wallet
```
echo "Send at least 30 tADA to hydra-node:"
echo $(cat credentials/hydra-node.addr)"\n"
```

```
echo "Send 1B of assets (NGN and WH) to reserve-wallet:"
echo $(cat credentials/reserve-wallet.addr)"\n"
```

Mint tokens
```
Open a new terminal window
cd app
export NEXT_PUBLIC_ENV="dev"
export NEXT_PUBLIC_HOST="localhost"
export NEXT_PUBLIC_PROTOCOL="http"
export NEXT_PUBLIC_PORT=":3000"
export
NEXT_PUBLIC_BLOCKFROST_API="https://cardano-preprod.blockfrost.io/api
/v0"
export NEXT_PUBLIC_BLOCKFROST_KEY="your-blockfrost-api-key"
npm install
npm run dev
Use a browser* to go to http://localhost:3000 and mint tokens for
reserve wallet above but must be sent in a single UTXO.
*Add Eternl wallet browser extension if not already installed
```

Inside the cardano-node container, cusomtize the protocol-parameters file for the hydra head
```
cardano-cli query protocol-parameters \
  | jq '.txFeeFixed = 0 |.txFeePerByte = 0 | .utxoCostPerByte = 0
|.executionUnitPrices.priceMemory = 0 |
.executionUnitPrices.priceSteps = 0' \
  > config/protocol-parameters.json

```
Inside the cardano-node container, confirm the UTXOs locked at the hydra head node and reserve wallet
```
echo "# UTxO of hydra-node"
```

```
cardano-cli query utxo --address $(cat credentials/hydra-node.addr)
--testnet-magic 1 | jq

echo "# UTxO of reserve-wallet"
cardano-cli query utxo --address $(cat
credentials/reserve-wallet.addr) --testnet-magic 1 | jq

# UTxO of hydra-node
{

"bc6c82ef126c927fc84db39b27e24df3a8b958b3df97d6a03d6c41702b68122a#0":
{
    "address":
"addr_test1vphjhjr7ravlzp0d4vnfd0awy8qq937l5zrfhzndpcguv9szmjven",
    "datum": null,
    "datumhash": null,
    "inlineDatum": null,
    "inlineDatumRaw": null,
    "referenceScript": null,
    "value": {
      "lovelace": 30000000
    }
  }
}
# UTxO of reserve-wallet
{

"83d2b6960363dba94d8f16e1c8b2f05da81174fa07689771f4a6a519db0ef58d#0":
{
    "address":
"addr_test1vq3c4vxj466jlgp2elgpzhu9gv35mck2pvxruk7utmrfa3qtkds90",
    "datum": null,
    "datumhash": null,
    "inlineDatum": null,
    "inlineDatumRaw": null,
    "referenceScript": null,
    "value": {
      "487f7138717d97cd76c1f0ed50a20e5e47b491ded3ddc5afc3d2ffd5": {
```

```
        "4e474e": 1000000000
      },
      "f6580a3e0847439839bc1a3e5c4f4ead39b175ec05fe4883009228ec": {
        "5748": 2000000000
      },
      "lovelace": 1198180
    }
  }
}

```
```
```

7. Hydra Head Setup

```
### Create the hydra head credentials
```
hydra-node gen-hydra-key --output-file credentials/hydra-head
```
### Start the hydra head
To do a clean start, `rm -r persistence` the directory
```
docker compose -f docker-compose.yaml up hydra-node-1
```

### Connect to the Hydra head using websocat
```
websocat ws://127.0.0.1:4001 | jq
```

### Initialize the hydra head using the websocat and wait for the
head to open
```
{ "tag": "Init" }

The following will be generated
```
```

```
{
  "headId":
"14cab222dcef7584fba3b630a1eaef996786fa80555511f836faf4cc",
  "parties": [
    {
      "vkey":
"2876f57b7a29d00d8633b083984016f455d9e901f61faff8371976fe9f6011f0"
    }
  ],
  "seq": 25,
  "tag": "HeadIsInitializing",
  "timestamp": "2025-11-13T03:02:42.338942949Z"
}

```

Commit tokens to Hydra head
```
echo "# UTxO of reserve-wallet"
cardano-cli query utxo \
  --testnet-magic 1 \
  --address $(cat credentials/reserve-wallet.addr) \
  --out-file reserve-wallet-commit-utxo.json

cat reserve-wallet-commit-utxo.json
{

"fa85e388259bbd9f88bc2c447beee3893542c006122a5e03c16b41977a78ad96#0":
{
        "address":
"addr_test1vzctfpxkpajgjc56y6znkw4dhw8vjatwfau3mj9tp5a67fgzu09fq",
        "datum": null,
        "datumhash": null,
        "inlineDatum": null,
        "inlineDatumRaw": null,
        "referenceScript": null,
        "value": {
```

```
"a036d93fc2ca6a0ba60d244166e6d396aefc5841a30973d19040b832": {
                "444444": 700
            },
            "lovelace": 1021470
        }
    }
}
```
Build a raw transaction matching the UTXO and address from above
```
cardano-cli conway transaction build-raw \
  --tx-in
fa85e388259bbd9f88bc2c447beee3893542c006122a5e03c16b41977a78ad96#0 \
  --tx-out
'addr_test1vzctfpxkpajgjc56y6znkw4dhw8vjatwfau3mj9tp5a67fgzu09fq +
700 a036d93fc2ca6a0ba60d244166e6d396aefc5841a30973d19040b832.444444'
\
  --fee 0 \
  --out-file tx.json
```
Create the asset-commit-utxo.json file using cborHex from tx.json and the utxo from reserve-wallet-commit-utxo.json
```
{
 "blueprintTx": {
   "type": "Tx ConwayEra",
   "description": "Ledger Cddl Format",
   "cborHex":
"84a300d9010281825820fca316319ac405f71d3e018562195df97252002852a978ba
f826ce618aed8ce000018182581d60571e32eaa2b1f7f5728eb62211ce8c9ff82d5b1
4ec3ae6a2a20965908200a1581c6b6a4188559f744cd18c9f905ee0990b34d881d92c
7169aad588259ea1434343431902580200a0f5f6"
}
,
 "utxo": {

"fca316319ac405f71d3e018562195df97252002852a978baf826ce618aed8ce0#0":
{
```

```
       "address":
"addr_test1vpt3uvh252cl0atj36mzyyww3j0lst2mznkr4e4z5gyktyqmedc0l",
       "datum": null,
       "datumhash": null,
       "inlineDatum": null,
       "inlineDatumRaw": null,
       "referenceScript": null,
       "value": {

"6b6a4188559f744cd18c9f905ee0990b34d881d92c7169aad588259e": {
               "434343": 600
           },
           "lovelace": 1021470
       }
   }
 }
```
In terminal window (in solar-village directory on host machine)
```
curl -X POST localhost:4001/commit \
  --data @asset-commit-utxo.json \
  > asset-commit-tx.json
```

Inside the cardano-node docker image
```
cardano-cli latest transaction sign \
  --tx-file asset-commit-tx.json \
  --signing-key-file credentials/reserve-wallet.sk \
  --out-file asset-commit-tx-signed.json

cardano-cli latest transaction submit \
  --testnet-magic 1 \
  --tx-file asset-commit-tx-signed.json

# Need to wait ~1 min...and will see the following using websocat
above
{
```

```
  "headId":
"14cab222dcef7584fba3b630a1eaef996786fa80555511f836faf4cc",
  "party": {
    "vkey":
"2876f57b7a29d00d8633b083984016f455d9e901f61faff8371976fe9f6011f0"
  },
  "seq": 48,
  "tag": "Committed",
  "timestamp": "2025-11-13T03:12:51.527778468Z",
  "utxo": {

"83d2b6960363dba94d8f16e1c8b2f05da81174fa07689771f4a6a519db0ef58d#0":
{
      "address":
"addr_test1vq3c4vxj466jlgp2elgpzhu9gv35mck2pvxruk7utmrfa3qtkds90",
      "datum": null,
      "datumhash": null,
      "inlineDatum": null,
      "inlineDatumRaw": null,
      "referenceScript": null,
      "value": {
        "487f7138717d97cd76c1f0ed50a20e5e47b491ded3ddc5afc3d2ffd5": {
          "4e474e": 1000000000
        },
        "f6580a3e0847439839bc1a3e5c4f4ead39b175ec05fe4883009228ec": {
          "5748": 2000000000
        },
        "lovelace": 1198180
      }
    }
  }
}
{
  "headId":
"14cab222dcef7584fba3b630a1eaef996786fa80555511f836faf4cc",
  "seq": 50,
  "tag": "HeadIsOpen",
  "timestamp": "2025-11-13T03:13:17.274346695Z",
```

```
  "utxo": {

"83d2b6960363dba94d8f16e1c8b2f05da81174fa07689771f4a6a519db0ef58d#0":
{
      "address":
"addr_test1vq3c4vxj466jlgp2elgpzhu9gv35mck2pvxruk7utmrfa3qtkds90",
      "datum": null,
      "datumhash": null,
      "inlineDatum": null,
      "inlineDatumRaw": null,
      "referenceScript": null,
      "value": {
        "487f7138717d97cd76c1f0ed50a20e5e47b491ded3ddc5afc3d2ffd5": {
          "4e474e": 1000000000
        },
        "f6580a3e0847439839bc1a3e5c4f4ead39b175ec05fe4883009228ec": {
          "5748": 2000000000
        },
        "lovelace": 1198180
      }
    }
  }
}

```
```
```
In terminal window (in solar-village directory on host machine)
```

## 8. Hydra Head Transactions

```
e tx.json \
  --signing-key-file credentials/reserve-wallet.sk \
  --out-file tx-signed.json

```
```
In terminal window (in solar-village directory on host machine)
```
```

```
curl -X POST 127.0.0.1:4001/transaction \
  --data @tx-signed.json

curl -s 127.0.0.1:4001/snapshot/utxo | jq
{

"c1f84507f2491f2b37de95a956c62db6a65da2d57dcd2ae937a9d0799e71ae3f#0":
{
    "address":
"addr_test1vqpdlgvqygnf3mvzvp0rv3mtrukjfsy9s65qflrwgzqsqecdurccm",
    "datum": null,
    "datumhash": null,
    "inlineDatum": null,
    "inlineDatumRaw": null,
    "referenceScript": null,
    "value": {
      "487f7138717d97cd76c1f0ed50a20e5e47b491ded3ddc5afc3d2ffd5": {
        "4e474e": 25
      }
    }
  },

"c1f84507f2491f2b37de95a956c62db6a65da2d57dcd2ae937a9d0799e71ae3f#1":
{
    "address":
"addr_test1vq3c4vxj466jlgp2elgpzhu9gv35mck2pvxruk7utmrfa3qtkds90",
    "datum": null,
    "datumhash": null,
    "inlineDatum": null,
    "inlineDatumRaw": null,
    "referenceScript": null,
    "value": {
      "487f7138717d97cd76c1f0ed50a20e5e47b491ded3ddc5afc3d2ffd5": {
        "4e474e": 999999975
      },
      "f6580a3e0847439839bc1a3e5c4f4ead39b175ec05fe4883009228ec": {
        "5748": 2000000000
```

```
      },
      "lovelace": 1198180
    }
  }
}

```
```

9. Hydra Head Decommit

```
```
cardano-cli conway transaction build-raw \
  --tx-in
c1f84507f2491f2b37de95a956c62db6a65da2d57dcd2ae937a9d0799e71ae3f#1 \
  --tx-out
'addr_test1vq58ln4m4v02uty8z4stdjzuj25h9xdfhef2zu3du7240lgtcrlk8+1198
180 + 999999975
487f7138717d97cd76c1f0ed50a20e5e47b491ded3ddc5afc3d2ffd5.4e474e +
2000000000
f6580a3e0847439839bc1a3e5c4f4ead39b175ec05fe4883009228ec.5748' \
  --fee 0 \
  --out-file decommit.json

cardano-cli latest transaction sign \
  --tx-body-file decommit.json \
  --signing-key-file credentials/reserve-wallet.sk \
  --out-file decommit-tx-signed.json

curl -X POST 127.0.0.1:4001/decommit \
  --data @decommit-tx-signed.json
```
```
```

```
# the following will appear in websocat after ~1 min
{
  "headId":
"14cab222dcef7584fba3b630a1eaef996786fa80555511f836faf4cc",
  "seq": 112,
  "signatures": {
    "multiSignature": [

"28d7a3b0c28b95180fc46533817d2084f8014fe10fc8bb2b6b5108c574b6715d3bd1
7cb6f20028c61182ba549b4f07647f4c6bed26d6337bb83f03d815fbac0f"
    ]
  },
  "snapshot": {
    "confirmed": [],
    "headId":
"14cab222dcef7584fba3b630a1eaef996786fa80555511f836faf4cc",
    "number": 2,
    "utxo": {

"c1f84507f2491f2b37de95a956c62db6a65da2d57dcd2ae937a9d0799e71ae3f#0":
{
        "address":
"addr_test1vqpdlgvqygnf3mvzvp0rv3mtrukjfsy9s65qflrwgzqsqecdurccm",
        "datum": null,
        "datumhash": null,
        "inlineDatum": null,
        "inlineDatumRaw": null,
        "referenceScript": null,
        "value": {
          "487f7138717d97cd76c1f0ed50a20e5e47b491ded3ddc5afc3d2ffd5":
{
            "4e474e": 25
          }
        }
      }
    },
    "utxoToCommit": null,
```

```
    "utxoToDecommit": {

"2f9d1fab6b9fb31f8b3ef823ec72c2055e0d1f6ae82a6782b77290d8a7fbc8b9#0":
{
        "address":
"addr_test1vq58ln4m4v02uty8z4stdjzuj25h9xdfhef2zu3du7240lgtcrlk8",
        "datum": null,
        "datumhash": null,
        "inlineDatum": null,
        "inlineDatumRaw": null,
        "referenceScript": null,
        "value": {
          "487f7138717d97cd76c1f0ed50a20e5e47b491ded3ddc5afc3d2ffd5":
{
            "4e474e": 999999975
          },
          "f6580a3e0847439839bc1a3e5c4f4ead39b175ec05fe4883009228ec":
{
            "5748": 2000000000
          },
          "lovelace": 1198180
        }
      }
    },
    "version": 0
  },
  "tag": "SnapshotConfirmed",
  "timestamp": "2025-11-13T03:30:36.271269345Z"
}
{
  "decommitTxId":
"2f9d1fab6b9fb31f8b3ef823ec72c2055e0d1f6ae82a6782b77290d8a7fbc8b9",
  "headId":
"14cab222dcef7584fba3b630a1eaef996786fa80555511f836faf4cc",
  "seq": 113,
  "tag": "DecommitApproved",
  "timestamp": "2025-11-13T03:30:36.271269915Z",
  "utxoToDecommit": {
```

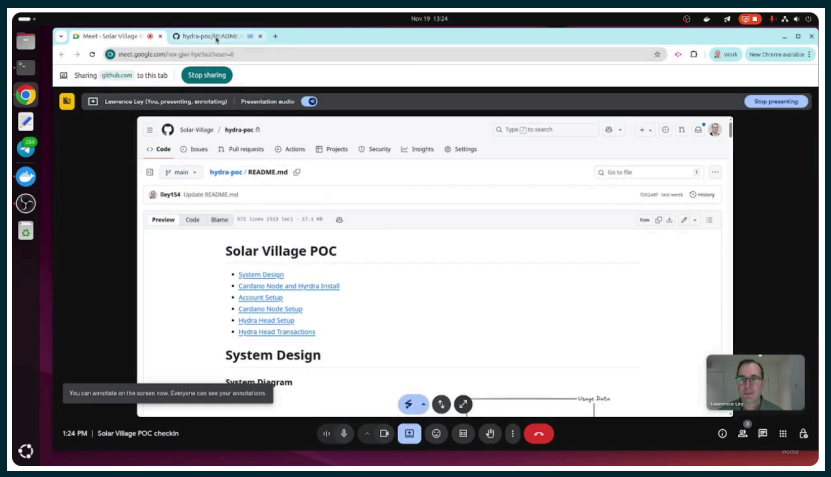
```
"2f9d1fab6b9fb31f8b3ef823ec72c2055e0d1f6ae82a6782b77290d8a7fbc8b9#0":
{
      "address":
"addr_test1vq58ln4m4v02uty8z4stdjzuj25h9xdfhef2zu3du7240lgtcrlk8",
      "datum": null,
      "datumhash": null,
      "inlineDatum": null,
      "inlineDatumRaw": null,
      "referenceScript": null,
      "value": {
        "487f7138717d97cd76c1f0ed50a20e5e47b491ded3ddc5afc3d2ffd5": {
          "4e474e": 999999975
        },
        "f6580a3e0847439839bc1a3e5c4f4ead39b175ec05fe4883009228ec": {
          "5748": 2000000000
        },
        "lovelace": 1198180
      }
    }
  }
}
{
  "distributedUTxO": {

"b6d0e826f3d4f02e0cb0f7fc009d7b507f41ecf1d02a68fca36e7b469f72c5aa#0":
{
      "address":
"addr_test1vq58ln4m4v02uty8z4stdjzuj25h9xdfhef2zu3du7240lgtcrlk8",
      "datum": null,
      "datumhash": null,
      "inlineDatum": null,
      "inlineDatumRaw": null,
      "referenceScript": null,
      "value": {
        "487f7138717d97cd76c1f0ed50a20e5e47b491ded3ddc5afc3d2ffd5": {
          "4e474e": 999999975
        },
```

```
        "f6580a3e0847439839bc1a3e5c4f4ead39b175ec05fe4883009228ec": {
          "5748": 2000000000
        },
        "lovelace": 1198180
      }
    }
  },
  "headId":
"14cab222dcef7584fba3b630a1eaef996786fa80555511f836faf4cc",
  "seq": 115,
  "tag": "DecommitFinalized",
  "timestamp": "2025-11-13T03:31:36.375320673Z"
}

```
```


10. Dependencies & Environment

Aiken CLI:

- Required for on-chain smart contract development and validation.
- Recommended: Latest stable release compatible with Conway era.

Cardano Node:

- Dockerized Cardano node running the Conway era, fully synced before Hydra operations.
- Network: Testnet (--testnet-magic 1).

Hydra:

- Hydra Node version: v1.1.0
- etcd version: v3.5.21 (used for Hydra head coordination)

Testnet:

- Cardano Preprod Testnet (used for funding, minting, and Blockfrost integration).

Off-chain services & tooling:
```

- Blockfrost API (Preprod) for wallet interaction and token minting
- Docker & Docker Compose for Cardano node, Mithril client, and Hydra setup
- Mithril client for fast chain snapshot bootstrap
- websocat for interacting with the Hydra head WebSocket API
- Node.js + npm for the local web app used to mint assets

11. Demo / Walkthrough

- Please see this short video demonstrating the prototype in action.
- https://www.youtube.com/watch?si=YFC93uzW8msRZUwc&v=f3fmP0moHTQ&feature=youtu.be



12. Remaining Issues and Next Steps

- Add a **clean-room setup script** (or Makefile) to automate node, Hydra, and etcd installation.
- Replace manual environment exports with a `.env.example` and documented environment variables.
- Formalize **wallet and key management** (key storage, backups, rotation, and security assumptions).
- Add explicit **network selection support** (Preview vs Preprod vs Mainnet) with guardrails to prevent misconfiguration.
- Validate Hydra flows with **multiple parties** instead of a single-node head.
- Automate **Hydra head lifecycle** (Init → Commit → Open → Close → Decommit) via scripts instead of manual `curl` calls.
- Define a **canonical asset schema** (policy IDs, asset names, decimals, semantics for NGN/WH).

- Add **invariants and safety checks** for commits, partial spends, and decommits.
- Introduce **error handling and retry logic** for Hydra WebSocket and HTTP interactions.
- Capture and persist **Hydra events and snapshots** for auditability and debugging.
- Add **basic tests** for off-chain transaction construction and submission.
- Validate behavior under **concurrent transactions** inside an open Hydra head.
- Measure and document **performance characteristics** (latency, throughput, head open/close times).
- Define a **failure recovery process** (node crash, Hydra restart, etcd loss).
- Clarify the **trust and threat model** (operator roles, custody assumptions, attack surfaces).
- Prepare a **minimal production architecture** (monitoring, logging, alerting).
- Identify what must change to move from **POC → Pilot** (UX, automation, reliability, security).

---

eof